

AEIS

Distributed Build Orchestration

Architektura działania systemu

Dokument roboczy opisujący dokładnie, jak ma działać rozproszona, równoległa produkcja kodu przez wiele serwerów i wielu agentów AI w ramach AEIS.

Cel: AEIS ma działać jak rozproszona firma programistyczna AI. Jeden orchestrator planuje i rozdziela pracę, wielu workerów koduje moduły niezależnie, a osobne warstwy integracji i governance pilnują jakości, kontraktów, rollbacku i zgodności z Księgą.

1. Cel systemu

System ma umożliwiać przejście od pomysłu i Księgi do gotowego baseline'u w sposób równoległy, skalowalny i kontrolowany. Zamiast jednego agenta, który sekwencyjnie buduje cały system, AEIS ma rozkładać projekt na moduły i workstreamy, przypisywać je do wielu workerów oraz stale integrować i walidować efekty pracy.

- Operator wprowadza pomysł, ograniczenia i Księgę.
- Canon Layer publikuje jedną prawdę o intencji, granicach i zasadach.
- Planning Layer rozbija projekt na moduły, kontrakty i zależności.
- Assignment Orchestrator rozdziela zakresy do workerów na wielu serwerach.
- Workerzy implementują tylko przypisane moduły i raportują status.
- Integration Layer stale sprawdza contract tests, integration tests, lint, typecheck i smoke tests.
- Governance Layer klasyfikuje zmiany, pilnuje evidence, rollbacku i approval flow.
- Operator Console pokazuje pełen stan budowy: worker fleet, moduły, blokery, decyzje i promocje zmian.

2. Zasada nadrzędna: Distributed Buildability by Contract

Każdy baseline projektowany przez AEIS musi być rozkładalny na niezależne moduły rozwijane równolegle przez wielu agentów lub zespołów. Równoległa implementacja jest dozwolona wyłącznie po zamrożeniu kontraktów, ownership, dependency graph i zasad integracji. Żaden worker nie może samowolnie zmieniać granic modułu ani publicznych interfejsów wpływających na inne moduły bez przejścia przez governance.

3. Architektura warstwowa

Warstwa	Kluczowe komponenty	Odpowiedzialność
A. Canon Layer	Canon Manager, Księga Store, Policy Store, Decision Class Registry	Jedno źródło prawdy: intencja, zasady, granice i canon snapshot.
B. Planning & Decomposition	Decomposition Engine, Dependency Analyzer, Ownership Planner, Contract Freeze Manager	Rozkład projektu na moduły, kontrakty, zależności i workstreamy.

C. Coordination	Assignment Orchestrator, Global Build State, Compact Generator, Status Aggregator	Przypisywanie zadań, stan budowy, heartbeaty, lokalne compacty.
D. Worker Build	Worker Runtime, repo sandbox, local tests, patch builder	Niezależna implementacja przypisanych modułów przez agentów na serwerach.
E. Integration & Validation	Integration Orchestrator, contract tests, integration tests, drift detector	Ciągła walidacja kompatybilności i jakości zmian.
F. Governance	Decision Classifier, Approval Engine, Rollback Planner, Evidence Pack Builder	Klasyfikacja zmian D0-D5, approval flow, rollback i evidence.
G. Operator / Control Plane	Dashboard Simple, Dashboard Pro, worker fleet view, build topology view	Widoczność, sterowanie, eskalacje i monitoring całej fabryki buildów.

4. Główne komponenty systemu

Canon Manager

Przechowuje Księgę, masterplan, polityki i publikowane canon snapshoty dla reszty systemu.

Decomposition Engine

Analizuje Księgę i masterplan, tworzy module map, dependency graph, ownership map i workstream plan.

Contract Registry

Przechowuje publiczne API, eventy, schematy, wersje i reguły kompatybilności. To jeden z najważniejszych komponentów całego systemu.

Module Registry

Lista modułów z polami: module_id, domena, owner, assigned_worker, status, dependencies, contract_version i lifecycle.

Assignment Orchestrator

Rozdziela zadania do workerów, bilansuje obciążenie i reaguje na blokery oraz zmiany priorytetów.

Worker Runtime

Uruchamiany na każdym serwerze. Pobiera assignment, compact, lokalny sandbox repo, wykonuje kodowanie, testy i zgłasza patch proposal.

Integration Orchestrator

Łączy propozycje zmian i uruchamia contract tests, integration tests, smoke tests, lint i typecheck.

Governance Engine

Określa klasę zmiany, sprawdza wymagane artefakty i decyduje, czy zmiana może zostać zmergowana lub czy wymaga review/Council/human gate.

Evidence Pack Builder

Tworzy paczkę dowodową zmiany: testy, uzasadnienie, decyzje, rollback plan, status approval i inne artefakty.

Observability Hub

Zbiera metryki z workerów, integracji i dashboardu.

Operator Console

Interfejs Simple i Pro do zarządzania build factory.

5. Model danych i obowiązkowe rejestry

- `module_registry.yaml` - mapa wszystkich modułów, statusów, ownerów i workerów.
- `contract_registry/` - publiczne API, eventy, payloads, wersje, compatibility rules.
- `assignments.json` - kto co robi, priorytety, blockery, ETA, aktualny krok.
- `integration_status.json` - stan testów, contract mismatches, drift, promotion readiness.
- `decision_log.md` - decyzje architektoniczne, uzasadnienia, approvals i wpływ na system.
- `worker_heartbeats.json` - heartbeat, obciążenie, stan lokalnych testów, zdrowie workerów.
- `recent_changes_compact.md` - skrócony stan systemu dla workerów.
- `evidence/<change_id>/` - pełne artefakty dowodowe dla zmian D3+ i wyżej.

6. Jak dokładnie działa przepływ od pomysłu do kodu

1. Ingestion: operator wprowadza pomysł, Księgę, ograniczenia i preferencje technologiczne.
2. Canon Snapshot: Canon Manager publikuje aktualną prawdę kanoniczną systemu.
3. Decomposition: Decomposition Engine rozbija system na kernel, moduły, kontrakty i workstreamy.
4. Contract Freeze: publiczne kontrakty, event taxonomy, ownership, dependency graph i integration rules zostają zamrożone.
5. Assignment: Assignment Orchestrator dzieli moduły między workerów zgodnie z workstream planem i zależnościami.
6. Worker Build: każdy worker pobiera lokalny compact, implementuje swój zakres, odpala lokalne testy i zgłasza patch proposal.
7. Integration Validation: Integration Orchestrator uruchamia contract tests, integration tests, lint, typecheck i smoke tests.
8. Governance Check: Decision Classifier określa klasę zmiany i włącza odpowiednie gate'y, review, rollback plan i evidence.
9. Merge / Promote: tylko zmiany zgodne z kontraktami, testami i governance mogą wejść do gałęzi integracyjnej i zostać promowane dalej.
10. Broadcast & Re-sync: zmiany kontraktów, wyników integracji lub decyzji są publikowane jako eventy; workerzy dostają nowe compacty i aktualizują lokalny stan.

7. Contract Freeze przed równoległą pracą

Parallel build bez frozen contracts jest zabroniony. Zanim workerzy zaczną niezależną implementację, system musi zamrozić granice. Bez tego każdy agent może zbudować inny model systemu i integracja stanie się chaotyczna.

- Zamrażane są: publiczne kontrakty modułów, event taxonomy, dependency graph, ownership, decision boundaries, security profile abstraction i integration acceptance rules.
- Zmiana tych elementów w trakcie build factory jest co najmniej D3.
- Zmiana wpływająca na wiele domen lub architekturę całego systemu może wejść w D4 lub D5.
- Worker nie może samowolnie edytować kontraktu należącego do innego modułu.

8. Model workera

Każdy worker to niezależna jednostka wykonawcza z własnym sandboxem, lokalnym checkoutem repo i własnym kluczem API do modelu.

Worker MUSI	Worker NIE MOŻE
pobrać assignment i compact	zmieniać cudzych publicznych kontraktów bez governance
pracować wyłącznie w przypisanym zakresie	czytać całego repo przy każdej zmianie, jeśli nie jest to potrzebne
odpalać lokalne testy	robić nieautoryzowanych zmian architektonicznych
publikować heartbeat i status	omijać integracji i gate'ów
zgłaszać patch proposal, a nie robić ciche merge'e	milcząco wprowadzać cross-module driftu

9. Worker compact zamiast pełnej pamięci wszystkiego

Worker nie powinien stale czytać całej historii projektu i całego repo. Zamiast tego otrzymuje compact - skrócony, aktualny, formalny stan potrzebny do pracy.

- jego moduły i aktualne zadania
- bezpośrednie zależności
- ostatnie ważne decyzje wpływające na jego zakres
- ostatnie zmiany kontraktów, które go dotyczą
- integration blockers wpływające na jego pracę
- priorytet i deadline
- lokalne acceptance criteria

10. Integracja jako obywatel pierwszej klasy

Integracja nie może być odłożona na koniec. Każda większa zmiana przechodzi przez continuous integration orchestration. To warstwa, która sprawdza nie tylko, czy moduł działa lokalnie, ale czy pasuje do reszty systemu.

- contract tests
- integration tests
- smoke tests
- lint
- typecheck
- drift detection
- bundle assembly
- promotion readiness

11. Governance w trybie rozproszonym

W build factory governance chroni system przed niekontrolowanym dryfem. Szczególnie istotne są zmiany cross-module, zmiany publicznych kontraktów i przebudowa topologii.

- Zmiana cross-module: minimum D3.
- Zmiana publicznego kontraktu: D3 lub wyżej.
- Zmiana dependency graph albo granic modułów: D3/D4.
- Zmiana globalnej topologii systemu albo pełnej organizacji build factory: D4/D5.

- Evidence pack i rollback plan są obowiązkowe dla odpowiednich klas zmian.

12. Topologia wdrożenia - wariant 5 serwerów

[Server 1] Control Plane

- Canon Manager
- Decomposition Engine
- Assignment Orchestrator
- Governance Engine
- Coordination API
- Postgres / NATS / Object Storage / Secrets

[Server 2] Worker A

- Core / Registry / Contracts

[Server 3] Worker B

- Cognitive / Memory

[Server 4] Worker C

- Execution / Skills / Connectors

[Server 5] Worker D + Integration

- Security / Monitoring / UI
- Contract tests / Integration tests / Candidate builds

To jest wariant minimalny. Działa, ale integracja i jeden z workerów dzielą zasoby na tym samym serwerze.

13. Topologia wdrożenia - wariant 8 serwerów (rekomendowany)

[Server 1] Canon & Planning

[Server 2] Coordination & Governance

[Server 3] Data & Event Backbone (Postgres, NATS, MinIO, Secrets)

[Server 4] Worker A - Core / Contracts / Registry

[Server 5] Worker B - Cognitive / Memory

[Server 6] Worker C - Execution / Skills / Connectors

[Server 7] Worker D - Security / Monitoring / Audit

[Server 8] Integration + Dashboard

To najlepszy balans pomiędzy prostotą a skalowalnością. Integracja jest odseparowana od workerów, a centralne usługi nie konkurują o zasoby z kodowaniem.

14. Topologia wdrożenia - wariant 10 serwerów

[Server 1] Canon Layer

[Server 2] Planning Layer

[Server 3] Coordination Layer

[Server 4] Governance Layer

[Server 5] Data / Event Backbone

[Server 6] Worker A - Core / Contracts

[Server 7] Worker B - Cognitive / Memory / Self-Model

[Server 8] Worker C - Execution / Connectors / Skills

[Server 9] Worker D - Security / Observability / Audit

[Server 10] Worker E - Dashboard / UI / Operator Console

Wariant 10-serwerowy daje wyraźnie większą specjalizację. Jeśli skala rośnie, integrację można wynieść jeszcze na osobny 11. host.

15. Mechanizm synchronizacji między workerami

- Pull: worker pobiera assignment, compact i zmiany kontraktów.
- Push: worker publikuje heartbeat, status, wyniki lokalnych testów i patch proposal.
- Event-driven update: gdy zmienia się kontrakt, dependency lub decyzja governance, odpowiednie workery dostają natychmiast aktualizację.
- Compact refresh: po istotnych zmianach system generuje nowy compact dla zainteresowanych workerów.
- Global Build State: dashboard i orchestrator widzą pełny stan, ale worker dostaje tylko ograniczony lokalny wycinek.

16. Repozytorium i model pracy

Najpraktyczniejszy model na start to monorepo z mocnym contract governance.

- Monorepo upraszcza integrację, wspólne kontrakty i dependency analysis.
- Każdy worker pracuje na osobnym branchu albo worktree.
- Worker nie merge'uje bezpośrednio do baseline.
- Każda zmiana jest patch propozalem przechodzącym przez integrację i governance.
- Folder ownership i contract ownership ograniczają chaos.

17. Model bezpieczeństwa i audytu

- Każdy worker ma osobny API key, worker_id i audit trail.
- Każda propozycja zmiany ma ślad: kto ją wygenerował, z jakiego serwera, na jakim zadaniu i na jakiej wersji compactu.
- Dostęp do sekretów i repo powinien być ograniczany per worker i per zakres.
- Governance i audit nie mogą być hostowane w tym samym runtime co niekontrolowane skrypty workerów, jeśli celem jest wyższa twardość systemu.

18. Antywzorce zakazane

11. Parallel Build Without Frozen Contracts
12. Silent Cross-Module Changes
13. Late Integration
14. Worker Full-Repo Blind Editing
15. Everyone Touches Everything
16. Contract Drift Without Governance
17. Merge Before Evidence
18. Shared Internal Structures Instead of Public Contracts

19. Minimalna kolejność wdrożenia systemu

19. Zbudować Canon Manager i spójny model Księgi / masterplanu.
20. Zbudować Module Registry i Contract Registry.
21. Dodać Dependency Graph i Ownership Map.
22. Zbudować Assignment Orchestrator.
23. Zbudować Worker Runtime z local sandbox i local tests.
24. Zbudować Integration Orchestrator.
25. Zbudować Governance Gate i Evidence Pack Builder.

26. Uruchomić Dashboard Pro z widokiem worker fleet, blokad, integracji i decyzji.
27. Dopiero potem skalować z 5 do 8-10 serwerów.

20. Kryteria sukcesu

- Wzrost liczby workerów realnie zwiększa throughput budowy.
- Konflikty kontraktów są wykrywane wcześniej, a nie na końcu projektu.
- Workerzy nie potrzebują pełnego kontekstu całego systemu do każdej zmiany.
- Cross-module drift nie przechodzi niezauważony.
- Integracja jest ciągła i automatyczna.
- Governance nie dusi pracy, ale zatrzymuje niebezpieczne zmiany.
- Operator widzi pełną topologię build factory i może szybko reagować.

21. Finalna rekomendacja

Jeśli AEIS ma kiedyś naprawdę budować duże systemy szybko, nie może być zaprojektowany jako jeden superagent. Musi być zaprojektowany jako: orchestrator + canon + contract system + wielu workerów + ciągła integracja + governance. Najrozsądniejszy start to architektura 8-serwerowa z monorepo, Postgres, event bus, object storage i osobnym hostem integracyjnym. W ten sposób system pozostaje prosty operacyjnie na początku, ale ma gotową ścieżkę do skalowania w prawdziwą rozproszoną firmę programistyczną AI.